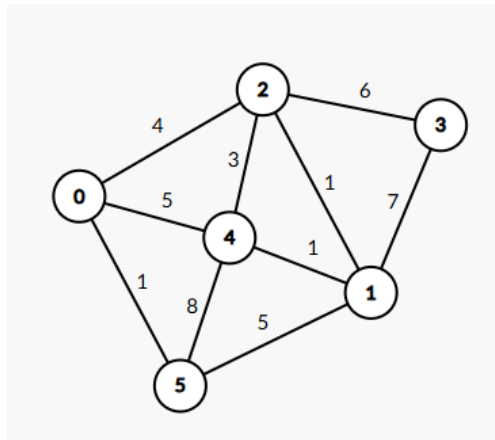


Aula 14 - Grafos (Caminho mínimo)

Um dos principais problemas dentro do estudo de Algoritmos em Grafos é o seguinte: dados dois vértices u e v em um grafo G , qual o **caminho mínimo** entre eles?

Adaptação para grafos ponderados

Esse tipo de problema normalmente é dado em um grafo **ponderado** (as arestas têm pesos), e o dito **caminho mínimo** é definido pelo caminho com a menor soma dos pesos das arestas nele, não necessariamente sendo o caminho com menos arestas.



Vamos analisar esse grafo durante essa aula.

Daí, precisamos adaptar a nossa estrutura de um grafo em C++ para suportar armazenar, também, os pesos nas arestas.

Lista de Arestas

Se antes tínhamos um grafo modelado apenas como uma lista de pares `vector<pair<int,int>>`, a adaptação é simples, só precisamos que cada par agora também tenha um terceiro elemento associado a ele. Ou seja, saímos de $e = \{u, v\}$ para $e = \{u, v, w\}$ em que w é o peso da aresta entre u e v .

Em código, temos inúmeras formas de fazer isso:

```
vector<pair<pair<int,int>, int>> grafo;  
vector<vector<int>> grafo(m, vector<int>(3)); // m é o número de arestas  
vector<tuple<int,int,int>> grafo;
```

No grafo anterior, ficaríamos com uma estrutura similar a:

```
vector<tuple<int,int,int>> grafo = {  
    {0, 2, 4},  
    {0, 4, 5},  
    {0, 5, 1},  
    {1, 4, 1},  
};
```

```

    {1, 5, 5},
    {2, 3, 6},
    {2, 4, 3},
    {4, 5, 8},
    {2, 1, 1},
    {3, 1, 7}
};

```

Matriz de Adjacência

Antes, tínhamos uma matriz $n \times n$ em que n é o número de vértices do grafo, com apenas 0's e 1's, sendo `matriz[u][v] == 1` se existe aresta entre u e v . Aqui, a adaptação é apenas substituir os 0's e 1's pelos pesos das arestas.

O grafo anterior fica:

G	0	1	2	3	4	5
0	0	0	4	0	5	1
1	0	0	1	7	1	5
2	4	1	0	6	3	0
3	0	7	6	0	0	0
4	5	1	3	0	0	8
5	1	5	0	0	8	0

Lista de Adjacência

Por fim, a lista de adjacência agora, além de, para cada vértice v , salvar todos os vizinhos de v , precisa também salvar o peso da aresta que liga v a cada vizinho.

Ou seja, cada elemento da lista deixa de ser um inteiro comum e passa a ser um par $\{\text{vizinho}, \text{peso}\}$.

Podemos fazer isso da seguinte forma:

```

vector<vector<pair<int,int>>> adj;
vector<pair<int,int>> adj[MAX]; // de forma similar a como fizemos na dfs/bfs

```

O grafo anterior fica algo como:

```

vector<pair<int,int>> grafo[7] = {
    {{2,4}, {4,5}, {5,1}}, // 0
    {{4,1}, {5,5}, {2,1}, {3,7}}, // 1
    {{0,4}, {3,6}, {4,3}, {1,1}}, // 2
    {{2,6}, {1,7}}, // 3
    {{0,5}, {1,1}, {2,3}, {5,8}}, // 4
    {{0,1}, {1,5}, {4,8}} // 5
};

```

Algoritmo de Dijkstra

O propósito desse algoritmo é, dado um vértice de origem, calcular a distância (soma dos pesos no caminho mínimo) desse vértice até todos os outros vértices do grafo. Esse algoritmo tem caráter guloso e possui inúmeras variações para suportar diversos tipos de subproblemas distintos, aprender a ideia do original para entender a ideia das variações é extremamente importante.

A grande ideia do algoritmo é: vamos sempre olhar para o vértice ainda não processado que tem a menor distância até a origem conhecida até agora. A partir desse vértice, vamos melhorar a distância de todos os seus vizinhos.

No grafo que citei anteriormente, vamos tratar o vértice 3 como a origem.

Intuição

Passo 1 - Inicialização

Primeiro, criamos um vetor que guarda as distâncias de todos os vértices até a origem. Inicialmente, a distância de todo mundo deve ser marcada como ∞ , exceto pela própria origem, que deve ser marcada como 0 (já que a distância da origem para ela mesma é 0).

Portanto: $\text{dist} = \{\infty, \infty, \infty, 0, \infty, \infty\}$.

Também precisaremos de um vetor de visitados, similar ao que tínhamos na bfs/dfs.

$\text{vis} = \{\text{false}, \text{false}, \text{false}, \text{false}, \text{false}, \text{false}\}$

Passo 2 - Loop de processamento

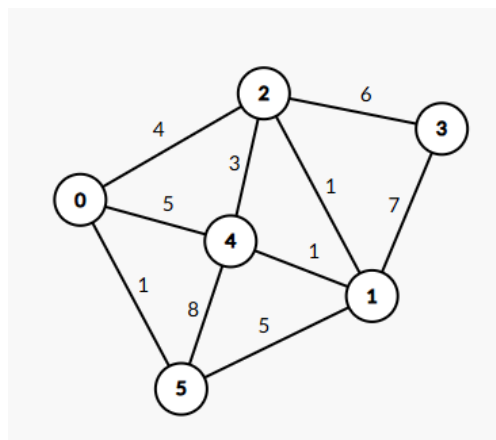
Entramos em um loop de escolher sempre o vértice que ainda não foi processado ($\text{vis}[v] = \text{false}$) com a menor distância (menor $\text{dist}[v]$), na primeira iteração, sempre é a origem. Daí, analisamos seus vizinhos, e realizamos uma operação chamada "relaxamento" em cada um deles.

O relaxamento consiste em verificar "passar por esse vértice melhora a distância total do vizinho dele?".

Matematicamente, para um vizinho v de u , $\text{dist}[v] > \text{dist}[u] + \text{peso}(u, v)$? Se sim, $\text{dist}[v] = \text{dist}[u] + \text{peso}(u, v)$. Ou seja, se a distância salva da origem até o vértice v é menor que a distância salva da origem até o vértice u mais a aresta entre u e v , vale a pena substituir o caminho que havíamos achado anteriormente para o vértice v pelo novo caminho, agora, vindo do vértice u .

Exemplo aplicado

Fica mais fácil de entender a ideia do algoritmo se aplicarmos em um grafo concreto.



Mesma figura de antes, repetida pra facilitar a visualização do algoritmo.

Primeira iteração

Na primeira iteração, olhamos para os vizinhos do 3 (já que ele é o que tem a menor distância salva, 0, e ainda não foi marcado como visitado).

No caso do 2, temos que $\text{dist}[2] > \text{dist}[3] + \text{peso}(3, 2)$, então, $\text{dist}[2]$ deixa de ser ∞ e passa a ser simplesmente 6.

Já no caso do 1, sua distância passa a ser 7. Assim, nossos vetores ficam dessa maneira →

v	dist[v]	vis[v]
0	∞	false
1	7	false
2	6	false
3	0	true
4	∞	false
5	∞	false

Segunda iteração

Agora, o próximo vértice não marcado como visitado e que tem a menor distância salva é o 2. Vamos olhar pros seus vizinhos.

O primeiro é o 0, que passa a ter distância 10. O segundo é o 4, que passa a ter distância 9. O terceiro e último é o 1, que não sofre alteração.

Nossos vetores ficam →

v	dist[v]	vis[v]
0	10	false
1	7	false
2	6	true
3	0	true
4	9	false
5	∞	false

Final

Vou me poupar de rodar o algoritmo inteiro, mas, no final da execução, é para estarmos com os vetores finais:

$\text{dist} = \{10, 7, 6, 0, 8, 11\}$ $\text{vis} = \{\text{true}, \text{true}, \text{true}, \text{true}, \text{true}, \text{true}\}$

Pseudocódigo

Notem que, no pseudocódigo, usamos uma estrutura que ainda não vimos: a fila de prioridade. Na parte em que “escolhemos o vértice com a menor distância”, poderíamos fazer um loop entre todos os vértices e procurar qual tem a menor distância e está marcado como $\text{vis} = \text{false}$, mas, a fila de prioridade entra pra fazer isso de forma automática.

O `.pop()` da fila de prioridade retorna sempre o **menor valor** dentro dela (no caso de uma fila mínima) ou o **maior valor** (no caso de uma fila máxima). A nossa servirá pra sempre retornar o vértice com a menor distância até agora.

Notem, também, que na fila de prioridade temos **pares** ao invés de inteiros comuns. Cada par representa um **vértice** e a distância salva da origem até aquele vértice.

DIJKSTRA(grafo, origem):

```
para cada vértice v:
    dist[v] = infinito
    visitado[v] = falso
```

```
dist[origem] = 0
```

```

fila_prioridade.push(0, origem)

enquanto fila não estiver vazia:

    (distAtual, u) = fila.pop()

    se visitado[u]:
        continue

    visitado[u] = verdadeiro

    para cada vizinho (v, peso):

        se dist[v] > dist[u] + peso:

            dist[v] = dist[u] + peso

            fila.push(dist[v], v)

```

Implementação em C++

```

#include <bits/stdc++.h>
using namespace std;

const int INF = 1e9;

int main() {

    int n, m;
    cin >> n >> m;

    // grafo em lista de adjacência
    vector<vector<pair<int,int>>> grafo(n);

    // leitura das arestas
    for(int i = 0; i < m; i++) {

        int u, v, peso;
        cin >> u >> v >> peso;

        grafo[u].push_back({v, peso});
        grafo[v].push_back({u, peso}); // se o grafo for direcionado, essa linha sai
    }

    int origem;
    cin >> origem;

```

```

vector<int> dist(n, INF);
vector<bool> visitado(n, false);

// (distancia, vertice)
priority_queue<
    pair<int,int>,
    vector<pair<int,int>>,
    greater<pair<int,int>>
> pq;

dist[origem] = 0;

pq.push({0, origem});

while(!pq.empty()) {

    // retirando o vértice com a menor distância armazenada
    pair<int,int> p = pq.top();
    int distAtual = p.first;
    int u = p.second;
    pq.pop();

    // se já foi processado, pulamos ele
    if(visitado[u])
        continue;

    visitado[u] = true;

    // olhando para os vizinhos e incluindo na fila de prioridade
    for(pair<int,int> p : grafo[u]) {
        int v = p.first();
        int peso = p.second();

        if(dist[v] > dist[u] + peso) {

            dist[v] = dist[u] + peso;

            pq.push({dist[v], v});
        }
    }
}

// saídas das distâncias
cout << "Menores distancias:\n";

for(int i = 0; i < n; i++) {

```

```

    cout << "Vertice " << i << ": ";

    if(dist[i] == INF)
        cout << "INF\n"; // isso ocorre se não há caminho da origem até i
    else
        cout << dist[i] << "\n";
}

}

```

Se quiserem o grafo que usei de exemplo nessa aula para testar:

```

6 10
0 2 4
0 4 5
0 5 1
1 4 1
1 5 5
2 3 6
2 4 3
4 5 8
2 1 1
3 1 7
3

```